Jetpack Compose, ViewModel, Notifications

Android Lecture 3

Simona Kurňavová

Today’s Agenda

- Jetpack Compose
- Material Design for Android
- ViewModel
- MVI, MVVM
- Notifications
- Q&A

Jetpack Compose

Jetpack Compose

- Modern toolkit for building native UI
- Intuitive Kotlin API
- UI in Kotlin instead of XML
- Emphasizes "what" over "how"
- Previews
- Any UI element in Compose is a Composable function = main building block
- Documentation

Jetpack Compose

- Reactive by nature (based on state)
- Does not recreate the whole UI when state change, just parts that need to change
- **Recomposition** = UI is updated based on state change

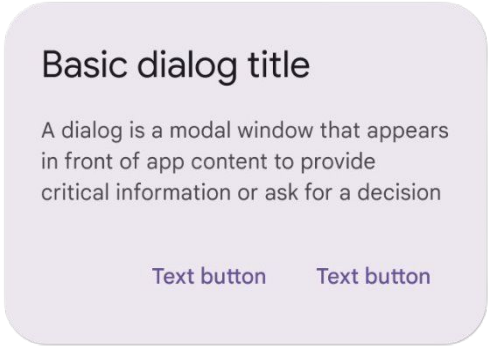
Jetpack Compose

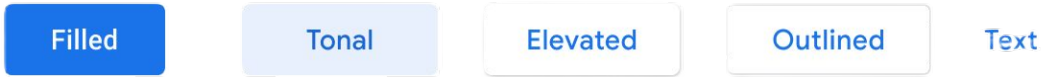
Key Concepts:

- *Reusability* - components should be generic
- *Modularity* - small building blocks of screen

Jetpack Compose: Basic components

- Text
- Image
- Button
- FloatingActionButton
- Spacer, Divider
- CircularProgressIndicator, LinearProgressIndicator
- AlertDialog, Popup
- …





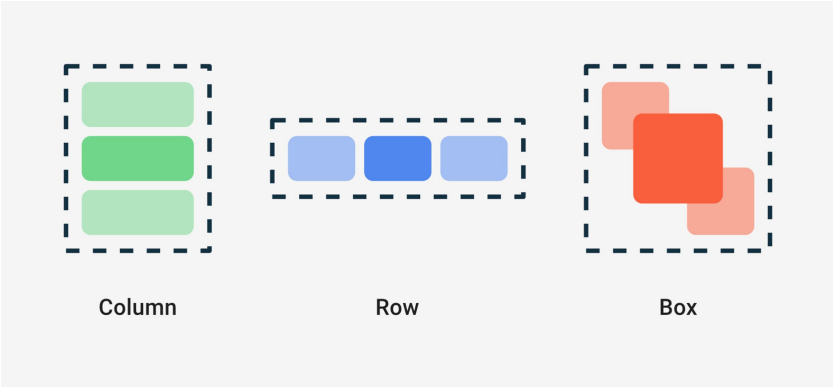
Jetpack Compose: Basic components

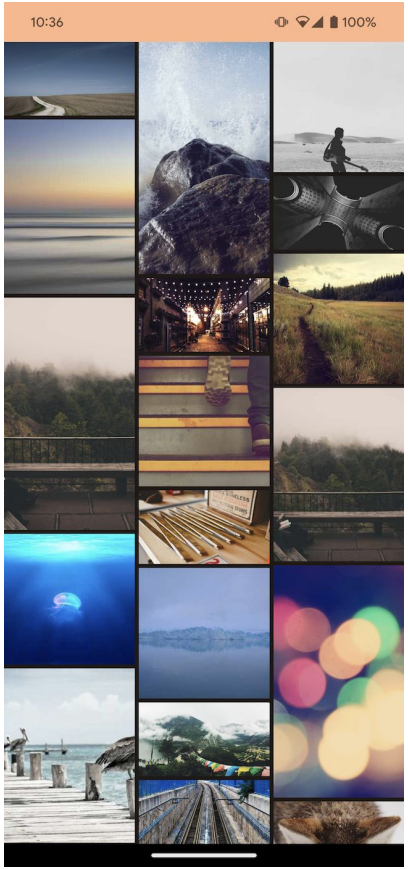
Simple Composables

```
@Composable
fun SimpleText() {
    Text("Hello Compose!")
}

@Composable
fun SimpleImage() {
    Image(
        painter = painterResource(id = R.drawable.my_image),
        contentDescription = "Example image"
    )
}

@Composable
fun SimpleButton() {
    Button(onClick = { /* do something */ }) {
        Text("Click me")
    }
}
```

Jetpack Compose: Layouts

- Box
- Column, Row
- LazyColumn, LazyRow
- LazyVerticalGrid, LazyHorizontalGrid
- LazyVerticalStaggeredGrid, LazyHorizontalStaggeredGrid (experimental)
- Scaffold





LazyVerticalStaggeredGrid

Jetpack Compose: Layouts

Simple Layouts

```
@Composable
fun LayoutExamples() {

    // Column: places children vertically
    Column {
        Text("First")
        Text("Second")
        Text("Third")
    }

    // Row: places children horizontally
    Row {
        Text("Left")
        Text("Center")
        Text("Right")
    }

    // Box: stacks children on top of each other (last one on top)
    Box {
        Text("Bottom layer")
        Text("Top layer")
    }
}
```

Jetpack Compose: Modifiers

- Layout, styling, interactivity
- Chainable
- **Order matters!**
- Examples:
  - padding()
  - background()
  - align()
  - clickable()
  - scrollable()
  - ...

```
1  @Composable
2  fun StyledText() {
3      Text(
4          text = "Hello, Compose!",
5          modifier = Modifier
6              .padding(16.dp)
7              .background(Color.LightGray)
8              .clickable {
9                  // Handle click action
10             },
11         color = Color.Black,
12         fontWeight = FontWeight.Bold,
13         textAlign = TextAlign.Center
14     )
15 }
```

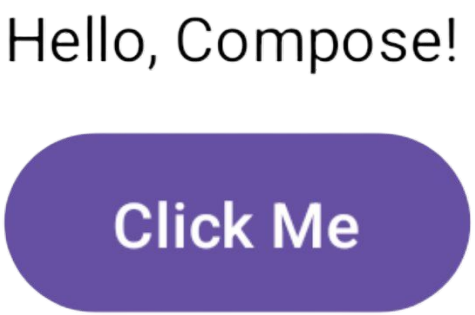
Jetpack Compose: Preview

- Documentation
- Allows viewing of Composables without launching application
- Preview is a standalone function
- Declared using **@Preview** annotation for **@Composable** function
- Defining Previews: dimensions, background, locale, light/dark theme, …
- **Interactive Preview:** allows interactions like animations, button clicks, etc.

Jetpack Compose: Preview

Kotlin basics

```kotlin
@Composable
fun SimpleComposeExample() {
    Column(
        modifier = Modifier.padding(16.dp)
    ) {
        Text("Hello, Compose!")

        Spacer(modifier = Modifier.padding(4.dp))

        Button(onClick = { /* Do something */ }) {
            Text("Click Me")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PreviewSimpleComposeExample() {
    SimpleComposeExample()
}
```

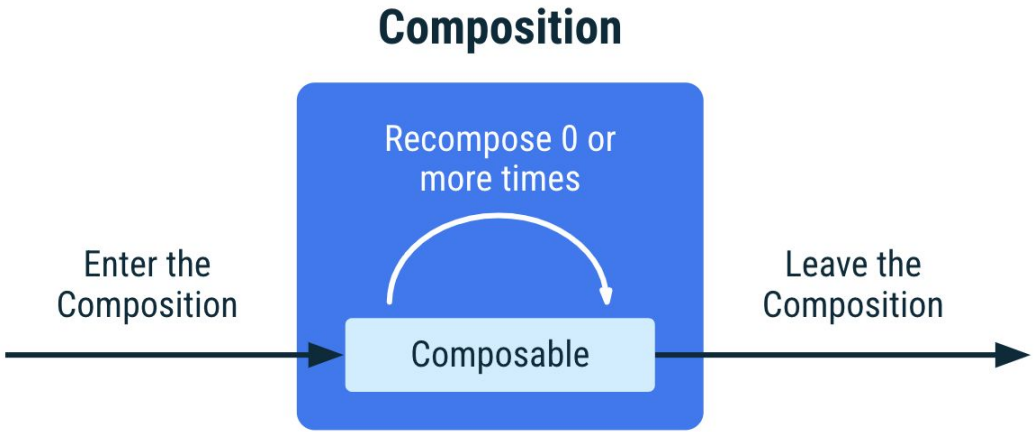


Jetpack Compose: When & how to use Preview

- Previews can serve as a development tool, but also as documentation of UI
- Previews should be **private** methods
- Types of Previews:
  - on screen level
  - on atomic component level
- You can use dummy data (strings, magic numbers) in Previews
- Use multiple previews for a screen/component where it makes sense:
  - dark/light theme
  - empty or corner case states (for example: empty list view)
  - alternate usages of component (for example: card with or without title/subtitle)

Jetpack Compose: Lifecycle

- **Key stages:** Composition, Recomposition, Disposal
- **Side-effect:** Change to the state of the app that happens outside the scope of a composable function
- **Effects:** Code that is triggered in response to changes in state or composition
  - *LaunchedEffect, SideEffect, DisposableEffect*



Jetpack Compose: State

- State = mutable data that can affect UI
- Changes in state trigger recomposition
- **MutableState:** observable type integrated with the compose runtime.
  - creation: mutableStateOf(value)
- **remember(params) { code block }**: remembers value in block across recompositions
  - params: optional; in case present -> re-execute code block when the value in parameters changes
- **rememberSaveable(params) { code block }**: remembers value in block across recompositions and configuration changes

Jetpack Compose: Declare State

- All these ways of declaring state are equivalent
- Use whichever one suits the task at hand

State Declaration

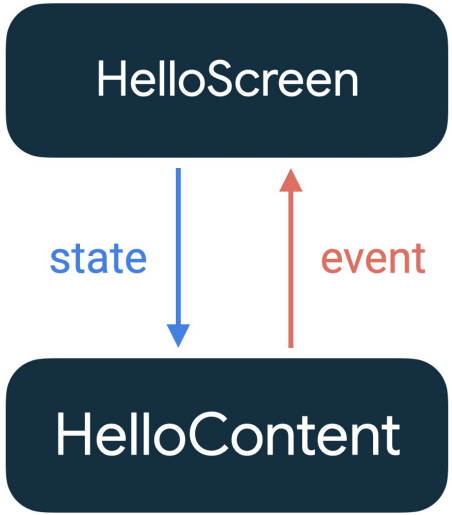
```
var mutableState = remember { mutableStateOf("") }
mutableState.value // access String value
mutableState.value = "new value" // change String value

var state by remember { mutableStateOf("") }
state // access String value
state = "new value" // change String value

val (state, setValue) = remember { mutableStateOf("") }
state // access String value
setValue("new value") // change String value
```

Jetpack Compose: Usage of State

```
@Composable
fun SimpleCounter() {
    val count = remember { mutableStateOf(0) }

    Button(onClick = { count.value++ }) {
        Text("Count: ${count.value}")
    }
}
```

Jetpack Compose: State hoisting

- **Careful Usage !**
- https://developer.android.com/develop/ui/compose/state#state-hoisting
- Pattern of moving state to a Composable's caller to make a composable *stateless*



Jetpack Compose: State hoisting

State Hoisting

```
@Composable
fun ToggleCaller() {
    var isOn by rememberSaveable { mutableStateOf(false) }

    Toggle(
        isOn = isOn,
        onValueChange = { isOn = it }
    )
}

@Composable
fun Toggle(
    isOn: Boolean,
    onValueChange: (Boolean) -> Unit
) {
    Button(onClick = { onValueChange(!value) }) {
        Text(if (isOn) "ON" else "OFF")
    }
}
```

Jetpack Compose: LaunchedEffect

- When `LaunchedEffect` enters the Composition -> it launches a coroutine with the block of code
- The coroutine will be cancelled if `LaunchedEffect` leaves the composition.
- If `LaunchedEffect` is recomposed with different **keys** -> existing coroutine will be cancelled and block of code will be launched in new coroutine

LaunchedEffect example

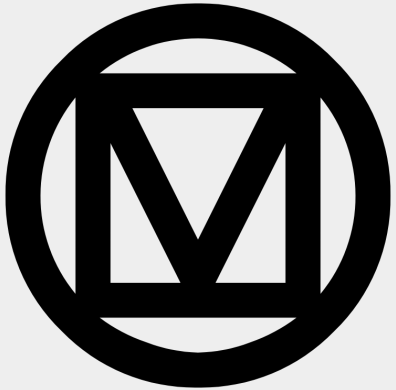
```
@Composable
fun UserProfileScreen(userId: String) {
    LaunchedEffect(userId) {
        Log.d("UserProfileScreen", "Loading data for userId = $userId")
        viewModel.loadUser(userId)
    }

    Text("User ID: $userId")
}
```

Navigation

- Navigation component
- Destinations are Composables
- *NavController*: Holds navigation graphs and provides API to move between Composables
- *NavHost*: Manages which composable is currently displayed based on NavController
- Documentation

Compose navigation

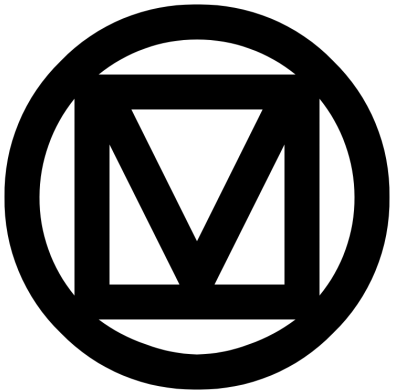
```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen() }
    composable("details") { DetailsScreen() }
}
```



How to design simple application with Material design

Material design

- Design language from Google
- Inspired by physical world (reflecting light, cast shadow)
- Cross-platform (Android, iOS, Flutter, web)
- Material components available as library
- https://www.material.io/
- Material 3: https://m3.material.io/styles



Material Theme in Compose

Key Principles:

- Color – color scheme of components
- Typography – styles of text
- Shape – defines components (rounded corners, elevation)

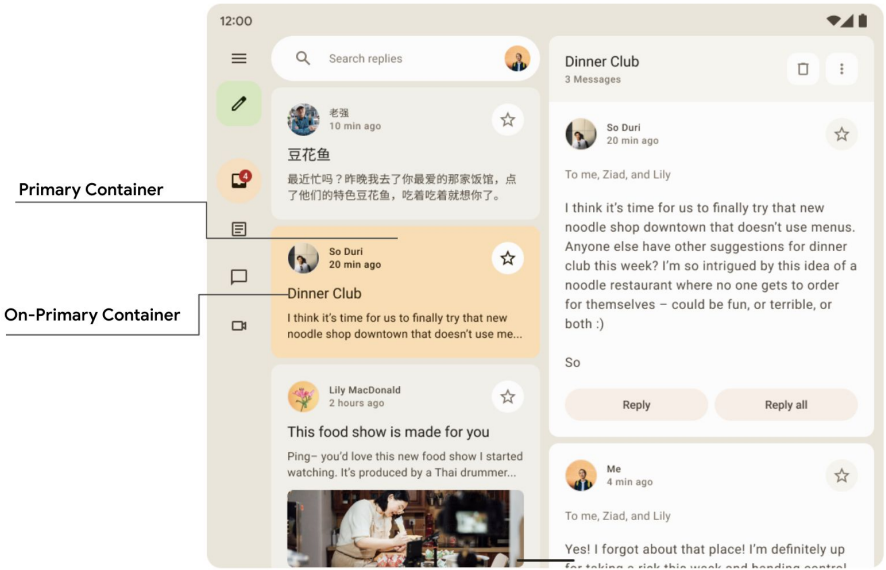
Material 3 in Jetpack Compose (Implementation):
https://developer.android.com/develop/ui/compose/designsystems/material3 (androidx.compose.material3)

Choosing color scheme

- Light & Dark theme color schemes
- Ensure contrast between text and background
- Pairs of color - primary & onPrimary, …
- https://m3.material.io/styles/color/system/overview

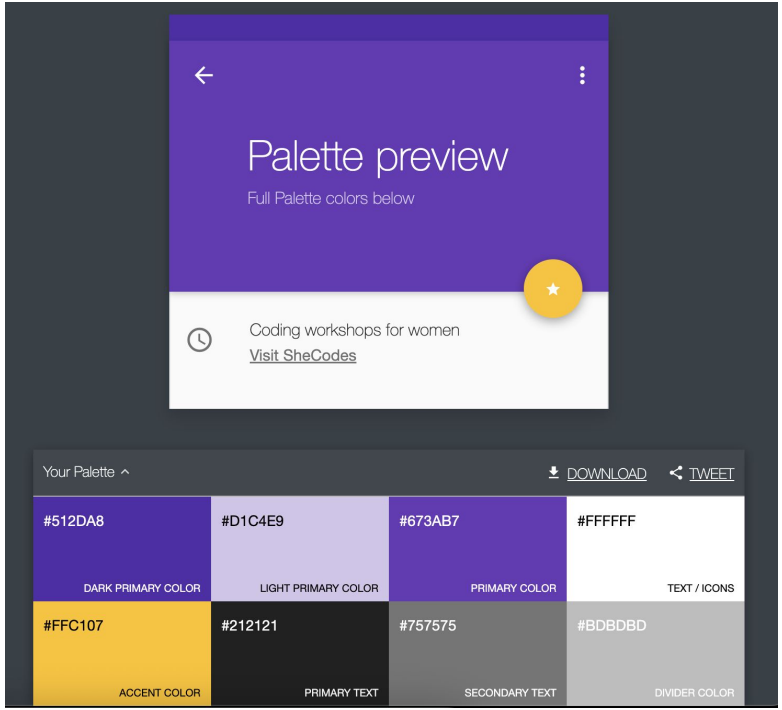
Light Theme

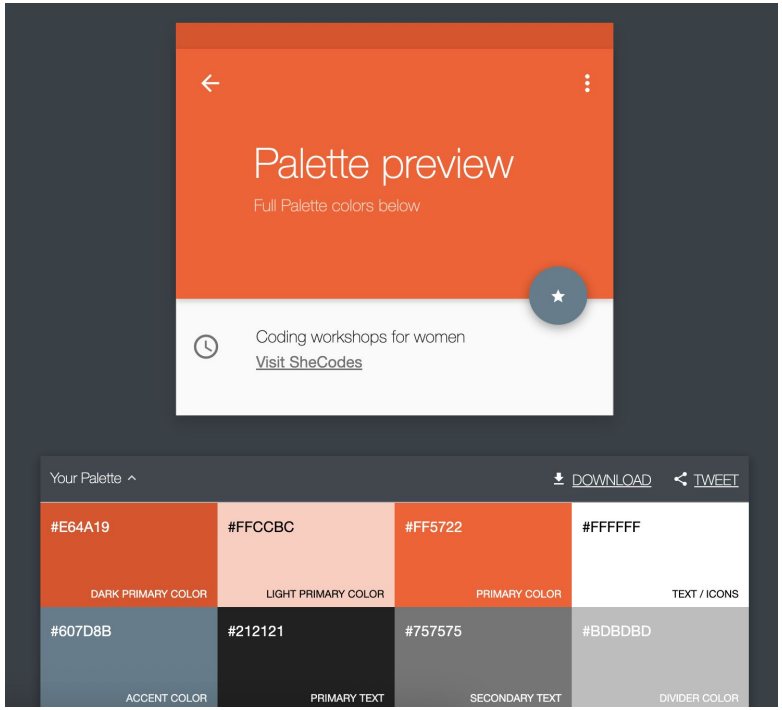
Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant



Choosing color scheme - Material Palette

- Color palette: https://www.materialpalette.com/





Color scheme: Definition

Material Theme - Color Definition

```
val light_primary = Color(0xFF476810)
val light_onPrimary = Color(0xFFFFFFFF)

val dark_primary = Color(0xFFACD370)
val dark_onPrimary = Color(0xFF213600)
...

private val LightColorScheme = lightColorScheme(
    primary = light_primary,
    onPrimary = light_onPrimary,
    ...
)
private val DarkColorScheme = darkColorScheme(
    primary = dark_primary,
    onPrimary = dark_onPrimary,
    ...
)
```

Typography: Definition

- Prefer semantic titles: headlineSmall, bodyLarge, labelMedium

Material Theme - Typography Definition

```
val myTypography = Typography(
    titleLarge = TextStyle(
        fontWeight = FontWeight.SemiBold,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    titleMedium = TextStyle(
        fontWeight = FontWeight.SemiBold,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.15.sp
    ),
    ...
)
```

Shapes: Definition

- Shapes of components

Material Theme - Shapes Definition

```
val myShapes = Shapes(
    extraSmall = RoundedCornerShape(4.dp),
    small = RoundedCornerShape(8.dp),
    medium = RoundedCornerShape(12.dp),
    large = RoundedCornerShape(16.dp),
    extraLarge = RoundedCornerShape(24.dp)
)
```

Material Theme: Definition

Material Theme - Definition

```
@Composable
fun MyTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme =
        if (!darkTheme) {
            LightColorScheme
        } else {
            DarkColorScheme
        }
    MaterialTheme(
        colorScheme = colorScheme,
        typography = myTypography,
        shapes = myShapes,
        content = content
    )
}
```

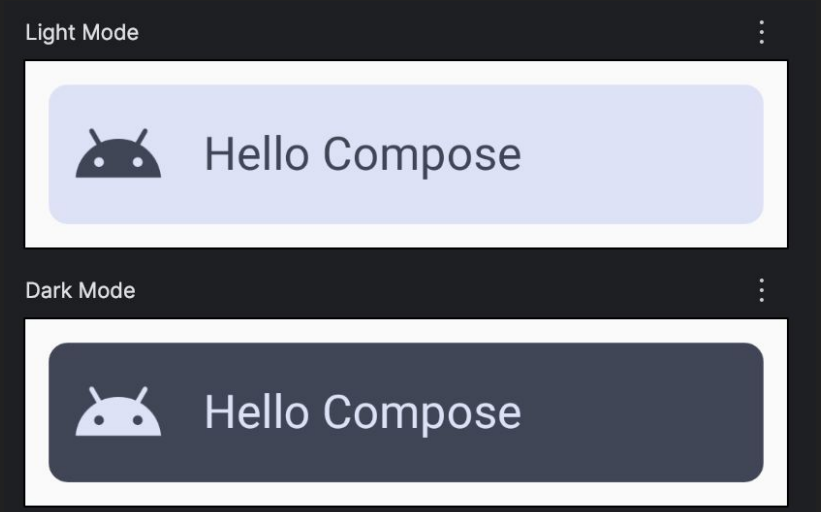
Material Theme: Usage

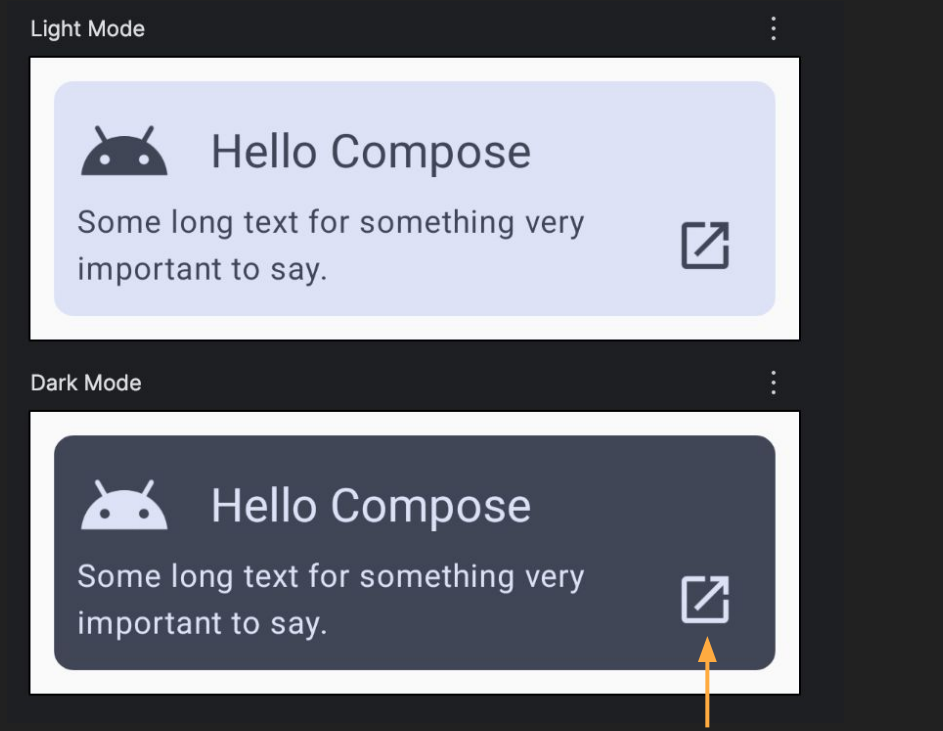
Material Theme - Usage

```
MyTheme {
    Box(
      shape = MaterialTheme.shapes.medium,
      color = MaterialTheme.colorScheme.surface
    ) {
        Text(
          text = "Hello",
          style = MaterialTheme.typography.bodyLarge
        )
    }
}
```

Jetpack Compose:
- Implement simple card

DEMO: Collapsed card vs. Expanded (after click)





ViewModel

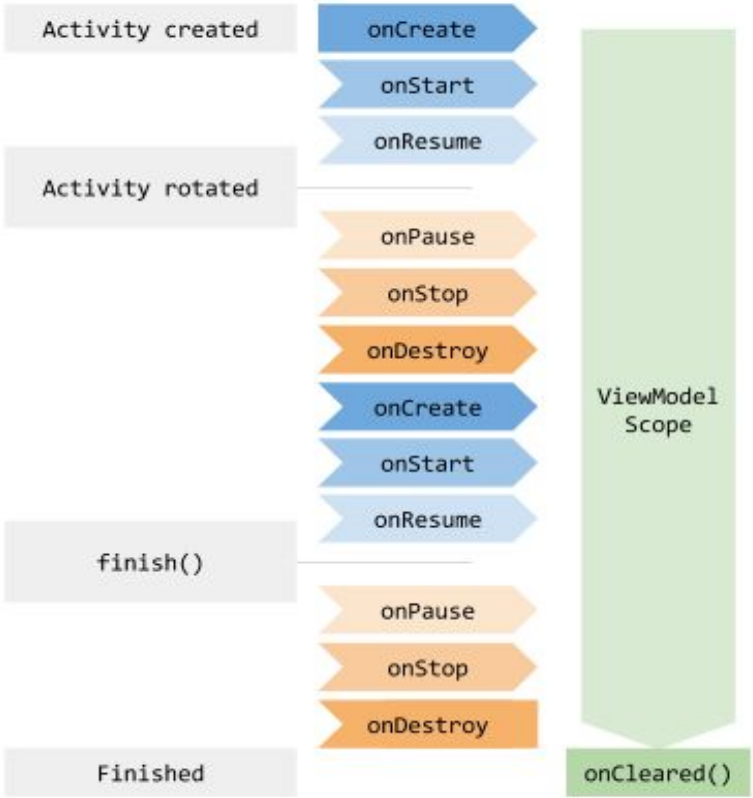
ViewModel

- Documentation
- Encapsulates and provides business logic for UI (activity, fragment, Composable)
- Lifecycle-aware
- Automatically retained during configuration changeå
- Part of Android Jetpack

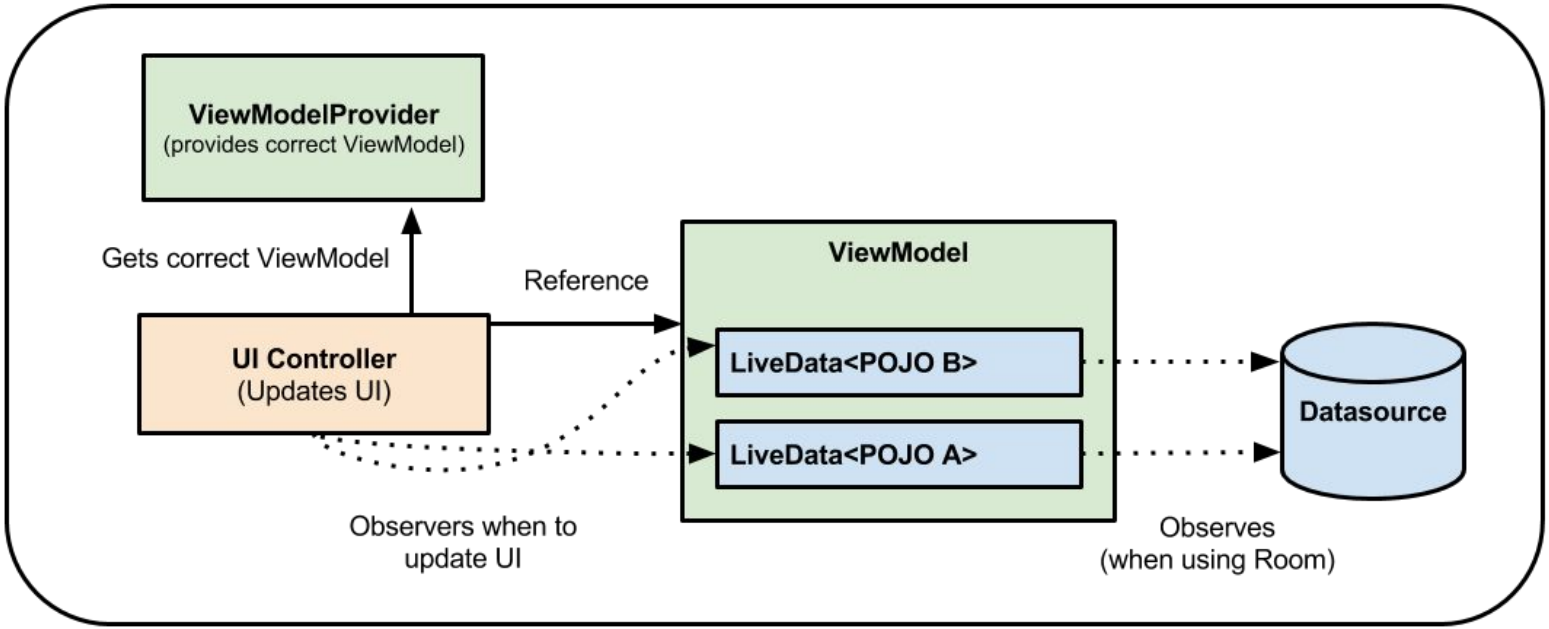
ViewModel Should never hold reference to View, Lifecycle or activity context (not even transitively)!

ViewModel: Lifecycle

- Illustration of lifecycle when activity is rotated
- ***onCleared***(): Called when ViewModel is going to be destroyed
  - *For Activity*: on finish
  - *For Fragment*: on detach



ViewModel - Architecture



ViewModel: Obtain

ViewModel in Fragment

```
class DemoViewModel : ViewModel() {
    private val _data = MutableStateFlow<String>("")
    val data = _data.asStateFlow()
}

class DemoFragment() : Fragment() {

   val demoViewModel: DemoViewModel by viewModels()

}
```

ViewModel in Compose

```
class DemoViewModel : ViewModel() {
    private val _data = MutableStateFlow<String>("")
    val data = _data.asStateFlow()
}

@Composable
fun DemoScreen(demoViewModel: DemoViewModel = viewModel())
{
   ...
}
```

Never create new instance of ViewModel by calling constructor!

ViewModelProvider.AndroidViewModelFactory

- Creates instances of ViewModel
- Default implementation uses
  - non-param constructor
  - Constructor which accept Application as the only parameter
- Often required with dependency injection framework

Flow, StateFlow, SharedFlow

- **Flow**:
  - Cold asynchronous data stream
  - Data is emitted only when collected
  - *Operators*: map, transform, takeWhile, filter, combine ...
  - *Usage*: One-time or continuous streams of data (network request, periodic update, …)
- **StateFlow**:
  - Hot stream
  - State container
  - *Usage*: Typically used to expose UI state in a ViewModel
- **SharedFlow**:
  - A hot flow that allows multiple collectors to receive the same emitted values.
  - *Usage*: Broadcasting one-time events, repetitive events

StateFlow Usage

StateFlow example

```
// ViewModel
class MyViewModel : ViewModel() {
    private val _state = MutableStateFlow("InitialState")

    val state: StateFlow<String> = _state.map { st ->
      st.uppercase() // Maps all values to upper case ones.
    }

    fun updateState(newState: String) {
        _state.value = newState
    }
}

...

// In a Composable function
@Composable
fun MyScreen(viewModel: MyViewModel) {
    val state by viewModel.state.collectAsState()

    Text(text = "Current State: $state")
}
```

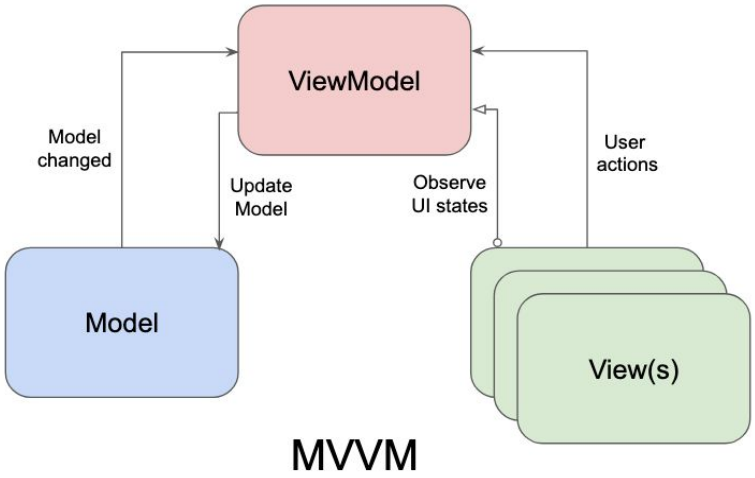
MVVM, MVI

MVVM (Model-View-ViewModel)

Model = represents data and business logic

View = represents the UI (Activity, Fragment, or Compose screen)

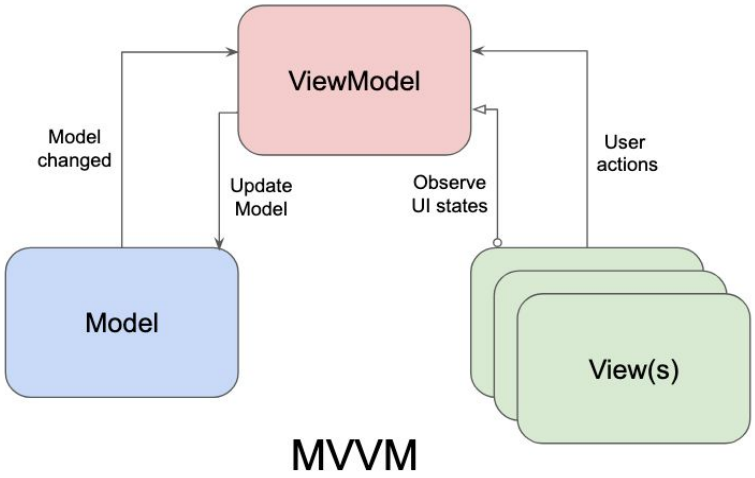
ViewModel = Acts as a bridge between View and Model.



MVVM (Model-View-ViewModel)

ViewModel responsibilities:

- Updates data against “model” (usually by calling Repository or Usecases - the responsibility of actually fetching data belong to different classes)
- Exposes state to Views as StateFlow or LiveData
- Handles user actions requested by View



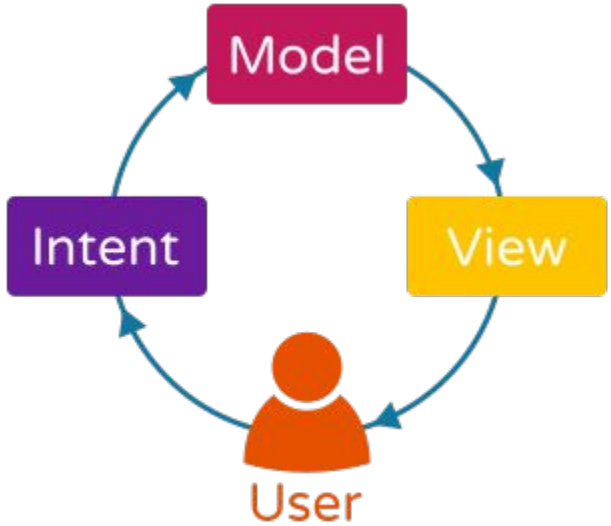
MVVM

MVI (Model-View-Intent)

Model = represents the state of the UI.

View = represents the UI (Activity, Fragment, or Compose screen)

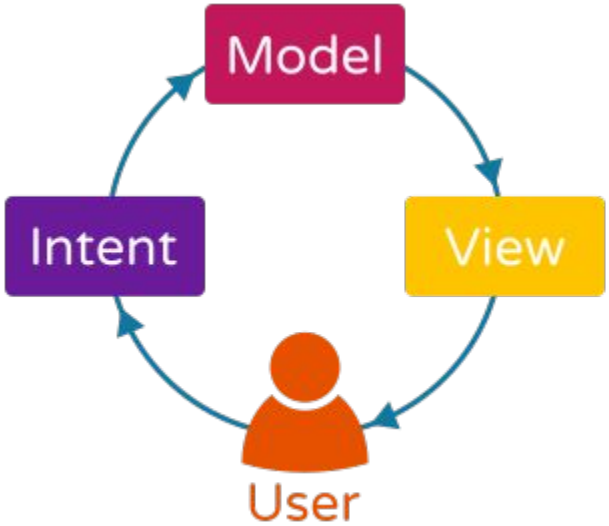
Intent = represents user actions or events.



MVI (Model-View-Intent)

ViewModel responsibilities:

- Exposes “Model” (state) to View
- Handles “Intent” send by View and updates Model accordingly



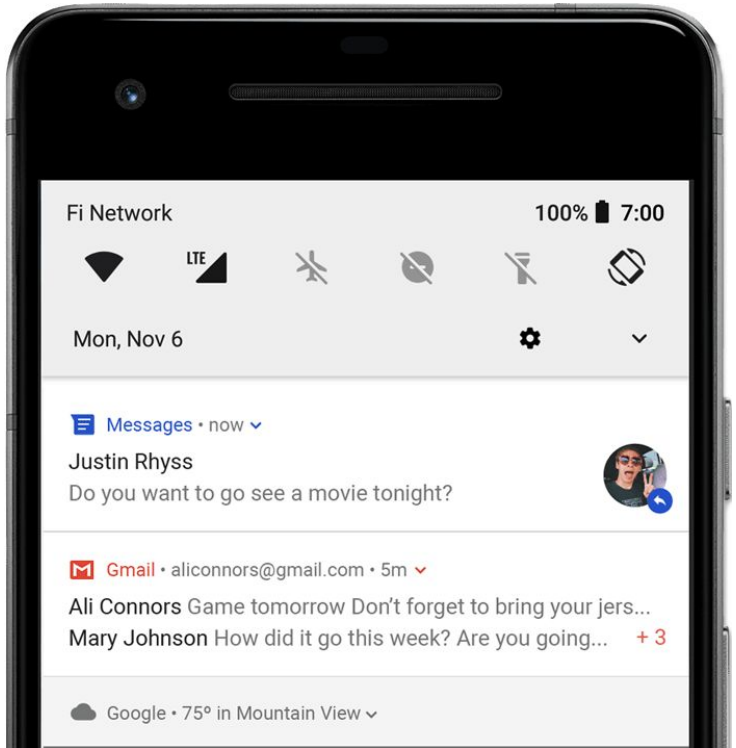
MVVM vs. MVI comparison

	MVVM	MVI
State	VM can expose **multiple** observable states	VM exposes only **one** observable state
Data Flow	**Possible Two Way**: VM exposes state and methods which UI can call to change state	**One Way**: View sends Intent -> VM processes and updates Model
User Actions	Calling of different methods exposed by VM	Always as “Intent” - unified form of communication (usually one method accepting Intent object)
UI updates	UI can be updated partially	UI is updated as a whole (one state)

Notifications

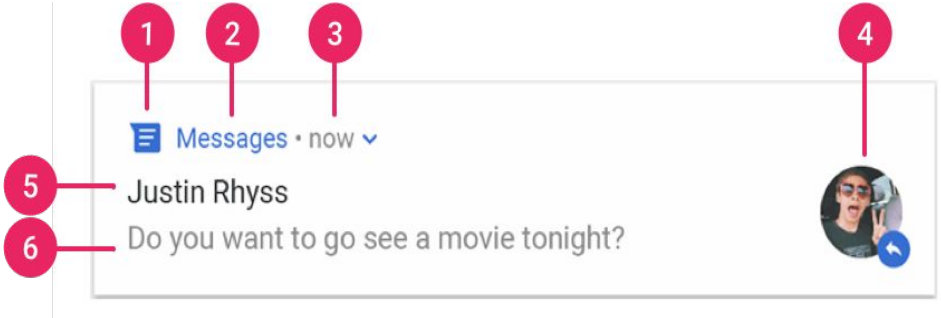
Notifications

- Notify user, when they are not using your app
- Appear on:
  - notification drawer
  - lock screen
  - status bar
  - badge on app icon
  - wearable devices



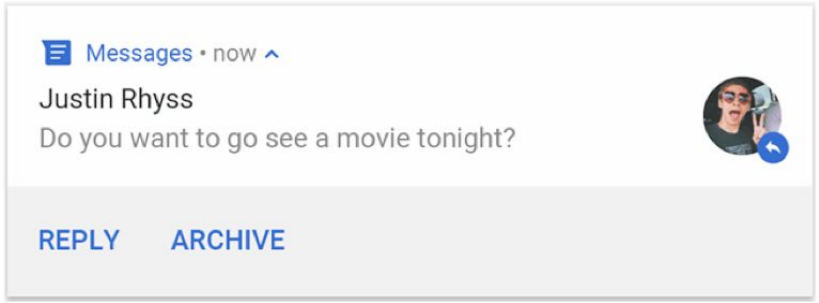
Notification

1. Small icon - mandatory parameter
2. App name - provided by system
3. Timestamp: provided by system,
   - override by setWhen()
   - Hide setShowWhen(false)
4. Large icon - optional
5. Title - optional
6. Text - optional



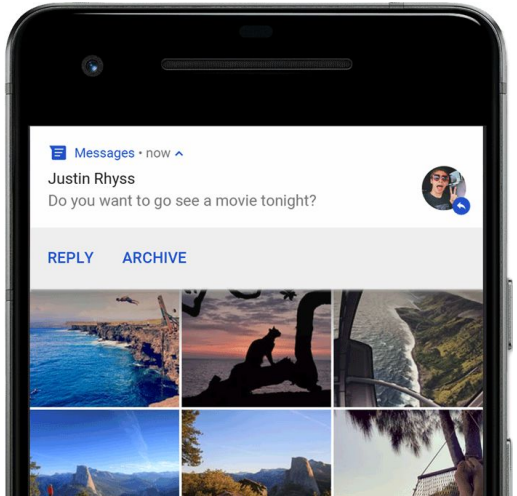
Notification actions

- Quick actions
- Inline reply action Android 7.0+



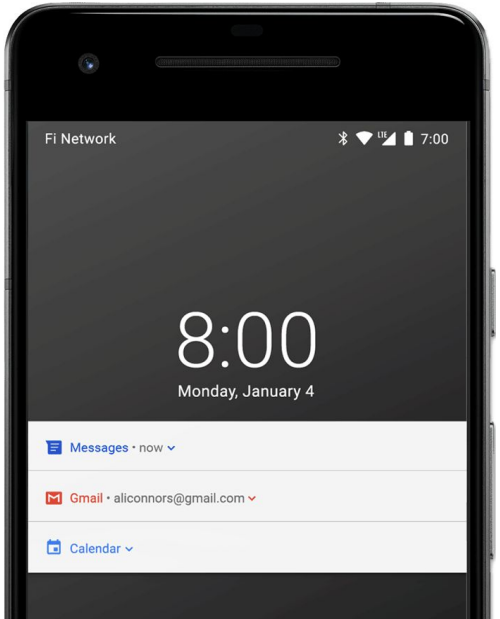
Heads-up notifications

- Heads-up notifications
  - Small floating window
  - Shows action buttons to handle action without leaving app
  - Only for high priority notifications
    - If the user’s activity is in full screen mode (app uses fullScreenIntent)
    - Notification has high priority and uses ringtones or vibrations
- Android 5.0+



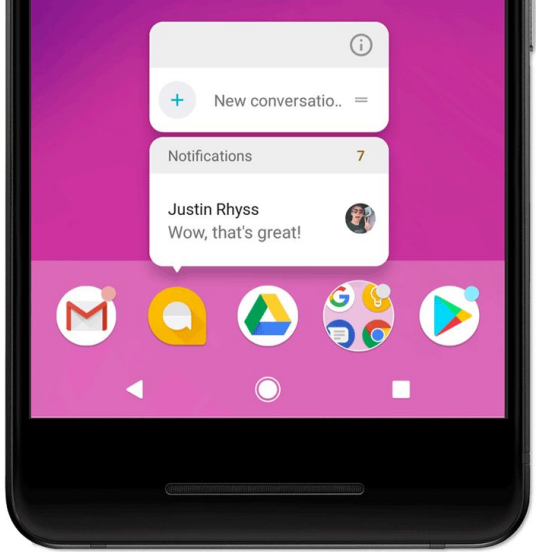
Lock screen notifications

- Possibility to set which informations when device is locked
  - VISIBILITY_PUBLIC - Shows full content
  - VISIBILITY_SECRET - Do not show at all
  - VISIBILITY_PRIVATE - Show icon and title, hide content
- Android 5.0+



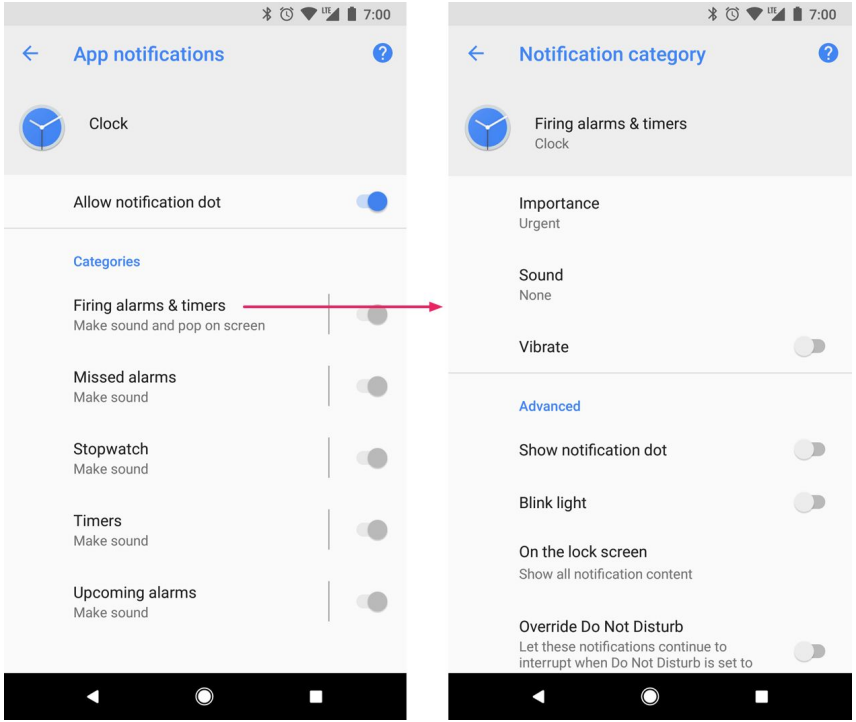
Notifications dots / badges

- Dots on launcher app icons
- Shows notification content on long click
- Android 8.0+



Notification channels

- Notification categories
- User can manage notifications in single category
- Override DnD
- System show number of deleted channels
- Android 8.0+



PendingIntent

An Intent you give to the system to execute later on your app's behalf, commonly used so notifications can open your app when tapped.

- Reference to a token describing the original Intent
- Possible to use even if the owning application is killed
- Used when the orignal Intent is needed later (Notification, Scheduler) for starting Activity, Service or sending Broadcast

Create notification

```
var builder = NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(R.drawable.notification_icon)
        .setContentTitle("My notification")
        .setContentText("Much longer text that cannot fit one line...")
        .setStyle(NotificationCompat.BigTextStyle()
                .bigText("Much longer text that cannot fit one line..."))
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
```

Create channel

```kotlin
private fun createNotificationChannel() {
    // Create the NotificationChannel, but only on API 26+ because
    // the NotificationChannel class is new and not in the support library
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val name = getString(R.string.channel_name)
        val descriptionText = getString(R.string.channel_description)
        val importance = NotificationManager.IMPORTANCE_DEFAULT
        val channel = NotificationChannel(CHANNEL_ID, name, importance).apply {
            description = descriptionText
        }

        // Register the channel within the system
        val notificationManager: NotificationManager =
            getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel)
    }
}
```

Setup notification action

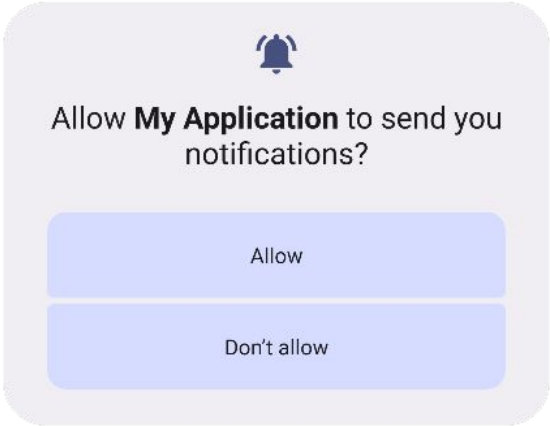
```
// Create an explicit intent for an Activity in your app
val intent = Intent(this, AlertDetails::class.java).apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
}

// Create the TaskStackBuilder
val resultPendingIntent: PendingIntent? = TaskStackBuilder.create(this).run {
    // Add the intent, which inflates the back stack
    addNextIntentWithParentStack(intent)
    // Get the PendingIntent containing the entire back stack
    getPendingIntent(0, PendingIntent.FLAG_UPDATE_CURRENT)
}

val builder = NotificationCompat.Builder(this, CHANNEL_ID)
        ....
        // Set the intent that will fire when the user taps the notification
        .setContentIntent(pendingIntent)
        .setAutoCancel(true)
```

Fire notification

```
with(NotificationManagerCompat.from(this)) {
    // notificationId is a unique int for each notification that you must define
    notify(notificationId, mBuilder.build())
}
```

Notifications: Changes

- **Android 12**:
  - No longer possible to control full content of notification (https://developer.android.com/about/versions/12/behavior-changes-12#custom-notifications)
- **Android 13**:
  - Requires runtime permission: POST_NOTIFICATIONS
  - Foreground services still available in Foreground service task manager, but not visible in notification drawer
  - Exemptions
    - Media sessions
    - App for managing phone calls



Thank you for attention

Feel free to leave feedback about this lesson: